Confidential mode v1 vs v2: who can ever see the node's API key

Protocol v1 is retired: the implementation now speaks only v2 (v1 is kept on the `v1-poc` branch). This page remains as the historical comparison.

The short version. Both versions hide prompts and answers from the node, pin the model and token cap, and bill from the usage the provider reports. They differ in one thing: **v1** keeps the node's provider API key from the client only as long as the client never sees the node's own encrypted traffic to the provider. **v2** removes that condition: client and node compute the TLS handshake keys jointly (a two-party computation, 2PC), so the client never holds the key that encrypts the record carrying the API key, even if it records the node's traffic. The price is bandwidth: about 22 MB node→client and 1.2 MB client→node per request, sent before the request exists.

	v1 (shipping)	**v2** (two-party handshake)
Node reads prompt or answer	no	no
Client can recover the node's API key	yes, **if** it (or anyone it colludes with) sees the node→provider ciphertext: a tap on the node's uplink, its hosting provider, a CDN edge	**no**: the record is encrypted under a key only the node has; recording it gives nothing
Model / token cap / billing	pinned by proofs, provider-reported usage	same
Who holds which TLS secret	client: all of them	client: handshake, response and body keys; node: handshake keys and the one key for its own record; nobody: the master secret
Key exchange	X25519	P-256 (the joint key share needs a full curve point)
Data per request, client and node (both directions)	about 6 MB	about 28 MB (23.3 MB 2PC: 22.1 MB to the client, 1.2 MB from it; 4.4 MB proofs)
Node upload per request	about 3 MB	about 24 MB (100 requests/min about 0.3 Gbit/s)
New cryptography in the trust base	ZK proofs	ZK proofs **and** a 2PC (node-only authenticated garbling, share conversion), not yet externally reviewed

Speed. Measured with real Venice (`deepseek-v4-flash`, 40 tokens), client and node on one host, medians of 5 sessions each unless noted:

	v1	v2
Cold: request start → body sent	0.73 s	1.2 s (2PC preprocessing 0.76 s; π_NK 0.06 s over the proof channel; one run)
Pre-warmed (armed through π_NK): claim → body sent	3 ms	5 ms
Suffix proof π_C2 (before the provider may start)	0.45 s	**at most 0.16 s** (component benchmark)
Pre-warm (arm) time per session	0.71 s	about 1.1 s (estimate)

Across a network link (from the component benchmarks, 2026-10 node-only garbling): the cold 2PC takes **3.1 s on a 100 Mbit/s symmetric link, 5.4 s on a 7/50 Mbit/s home link and 7.4 s on 2/50 Mbit/s** (80 ms RTT); its online part is about 0.5 s. In exchange π_C1 (1.6 s) disappears and the suffix proof drops from 1.6 s to 1.2 s. Cold, v2 is about a second slower than v1 on a fast link and several seconds slower on a slow uplink. **Pre-warmed, v2 is faster** (only the suffix proof is on the path, and it is cheaper), so v2 wants pre-warm: the tables cannot be reused, one set per request, but they are input-independent and can be built and sent ahead.

Security for the client (prompt and answer). v1: nothing new; the client alone runs standard TLS. v2: the client still verifies the provider's certificate itself and alone holds the body and response keys, but those keys now come out of a 2PC with the node. The 2PC is built so that a cheating node is caught before any key is used (the node garbles, its wire masks are authenticated to the client and every gate is checked); the most a cheating node can learn is **about two bits** (which of a few abort points was hit) about fresh, single-use shares, and the session then aborts. The node can write more than one record in its epoch (v1 gave it material for exactly one), but that epoch carries none of the client's data, and in v1 the client could not see the node's wire bytes either.

Security for the node (API key). v1: the key is safe only on a trusted egress path; one leak exposes it to anyone until rotated. v2: safe against a client that records everything; a cheating client (the evaluator) learns nothing beyond its own outputs and cannot change the node's, and cannot substitute its own key share (the node relays only a ClientHello carrying the joint share). v2 reveals nothing extra to the client and drops π_C1. In both versions the node must trust the provider with its key, as it does today.

Choosing. v1 for nodes whose upstream path is under their control and for low-bandwidth clients without pre-warm. v2 for nodes on shared or untrusted infrastructure, or with valuable keys: it turns P1 from an operational assumption into a cryptographic property, for about 22 MB per request to the client and 1.2 MB from it. A node runs one version; clients pick it up from the signed offer.